

OOP IN JAVA

Instructor:



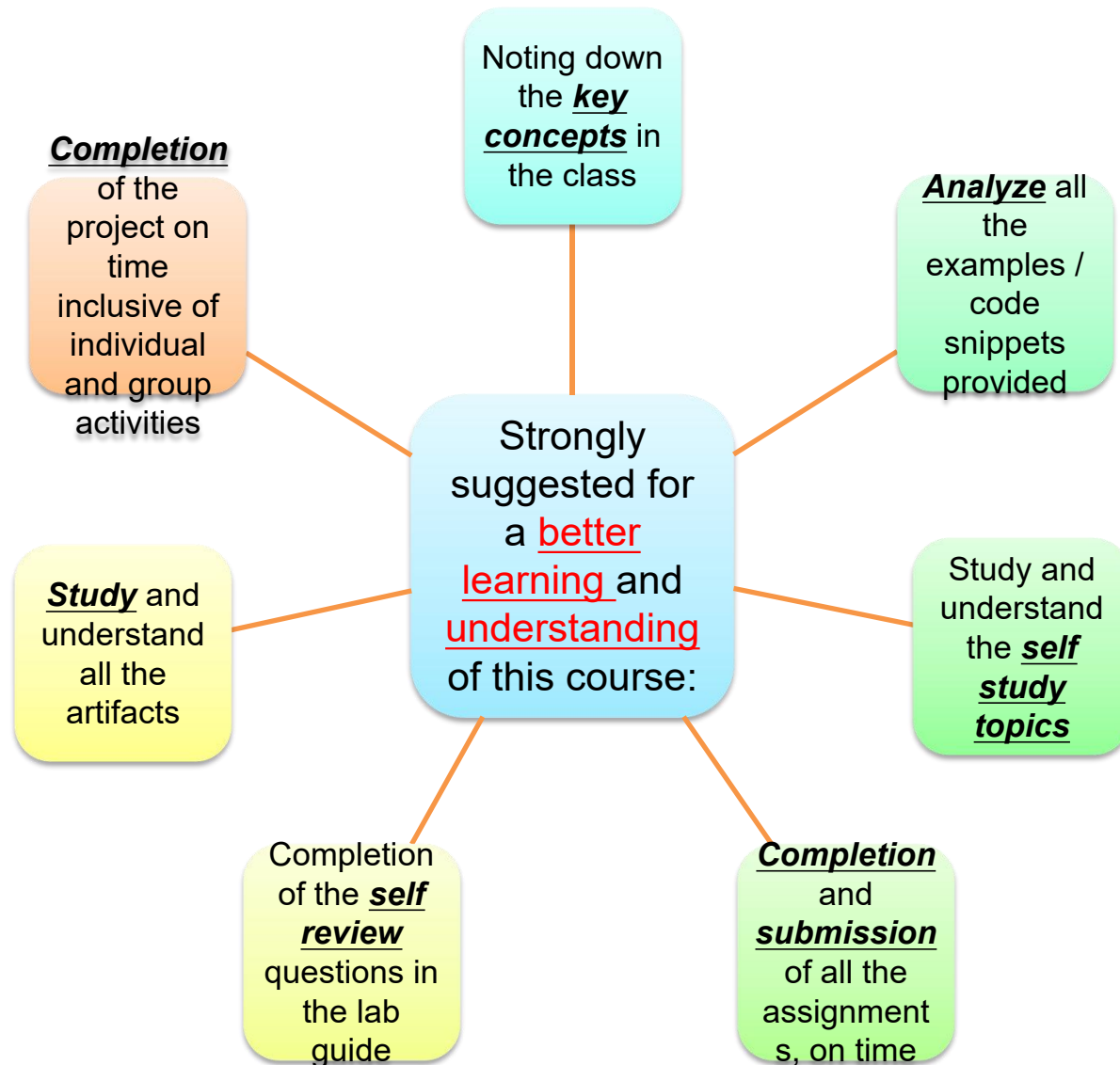
◇ Principles of OOP

❖ Inheritance

❖ Polymorphism

❖ Encapsulation

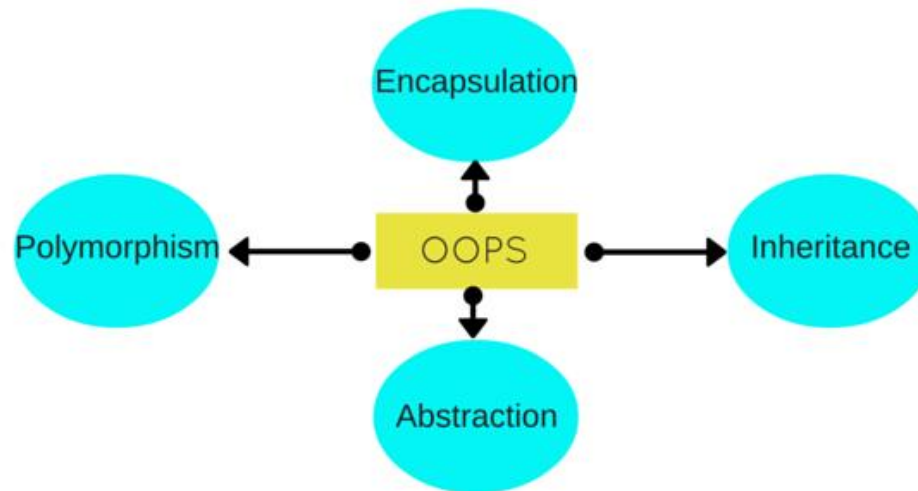
❖ Abstraction



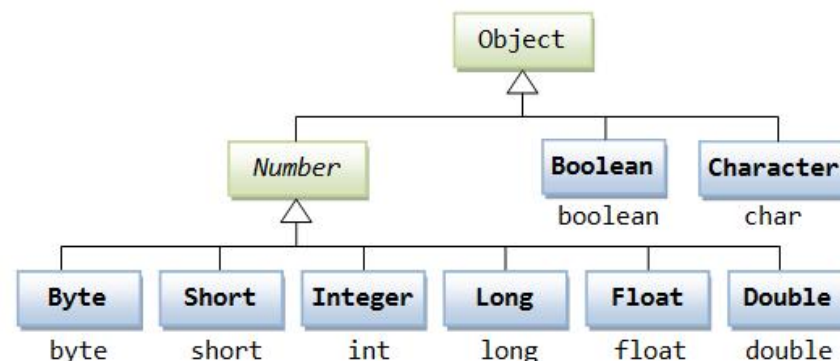
Section 1

Principles of OOP

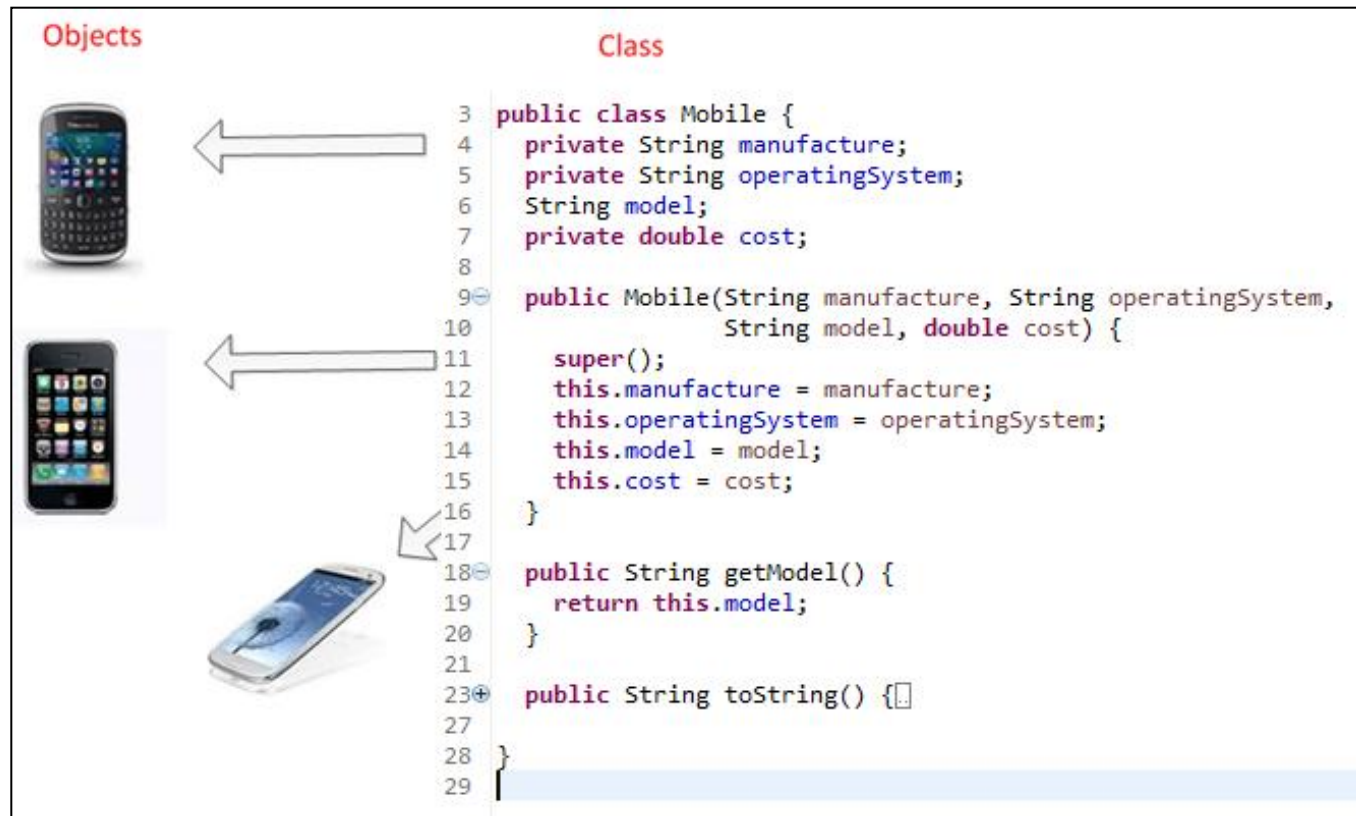
- ❖ **Object - Oriented Programming** system (OOPs) is a programming paradigm based on the concept of “**objects**” that contain data and methods.
- ❖ The primary purpose of object-oriented programming is to **increase the flexibility and maintainability** of programs.
- ❖ Java is an **object oriented language** because it provides the features to implement an object oriented model.
- ❖ These features includes **Abstraction, Encapsulation, Inheritance** and **Polymorphism**.



- ❖ When one object acquires all the **properties** and **behaviors** of a **parent object**, it is known as **inheritance**. It provides code reusability.
- ❖ Inheritance provides the idea of **reusability** of code and each **subclass** defines only those features that are unique to it, the rest of the features can be inherited from the **parent class**.
 - ✓ Inheritance is a process of defining a new class based on an existing class by extending its common data members and methods.
 - ✓ Inheritance allows us to reuse of code, it improves reusability in your java application.
 - ✓ The parent class is called the **base class** or **super class**. The child class that extends the base class is called the derived class or **sub class** or **child class**.



- ❖ You can look into the following example for inheritance concept.
- ❖ **Mobile** class:



- The **Mobile** class extended by other specific class like **Android** and **Blackberry**.

- **Android class:**

```
3 public class Android extends Mobile {
4
5     // Constructor to set properties/characteristics of object
6     public Android( String manufacture, String operatingSystem,
7                     String model, double cost) {
8         super(manufacture, operatingSystem, model, cost);
9     }
10
11    // Method to get access Model property of Object
12    public String getModel() {
13        return "This is Android Mobile- " + model;
14    }
15 }
16
```

- **Blackberry class**

```
3 public class Blackberry extends Mobile {
4
5     // Constructor to set properties/characteristics of object
6     public Blackberry(String manufacture, String operatingSystem,
7                       String model, double cost) {
8         super(manufacture, operatingSystem, model, cost);
9     }
10
11    public String getModel() {
12        return "This is Blackberry-" + model;
13    }
14 }
15
```

- ❖ If *one task is performed by different ways*, it is known as polymorphism.
 - ✓ Use **method overloading** and **method overriding** to achieve polymorphism.

```
3 public class Animal {
4     public void makeNoise() {
5         System.out.println("Some sound");
6     }
7 }
8
9 class Dog extends Animal {
10    public void makeNoise() {
11        System.out.println("Bark");
12    }
13 }
14
15 class Cat extends Animal {
16    public void makeNoise() {
17        System.out.println("Meawoo");
18    }
19 }
20
```

```
3 public class AnimalTest {
4
5     public static void main(String[] args) {
6         Animal a1 = new Cat();
7         a1.makeNoise(); // Prints Meowoo
8
9         Animal a2 = new Dog();
10        a2.makeNoise(); // Prints Bark
11    }
12 }
13
```

❖ *Hiding internal details and showing functionality* is known as abstraction.

✓ Use **abstract class** and **interface** to achieve abstraction.

```
3 public abstract class VehicleAbstract {
4     public abstract void start();
5
6     public void stop() {
7         System.out.println("Stopping Vehicle in abstract class");
8     }
9 }
10
11 class TwoWheeler extends VehicleAbstract {
12     @Override
13     public void start() {
14         System.out.println("Starting Two Wheeler");
15     }
16 }
17
18 class FourWheeler extends VehicleAbstract {
19     @Override
20     public void start() {
21         System.out.println("Starting Four Wheeler");
22     }
23 }
24 }
```

```
3 public class VehicleAbstractTest {
4
5     public static void main(String[] args) {
6         VehicleAbstract my2Wheeler = new TwoWheeler();
7         VehicleAbstract my4Wheeler = new FourWheeler();
8         my2Wheeler.start(); // Prints "Starting Two Wheeler"
9         my2Wheeler.stop(); // Prints "Stopping Vehicle in abstract class"
10        my4Wheeler.start(); // Prints "Starting Four Wheeler"
11        my4Wheeler.stop(); // Prints "Stopping Vehicle in abstract class"
12    }
13 }
14
15 }
16 }
```

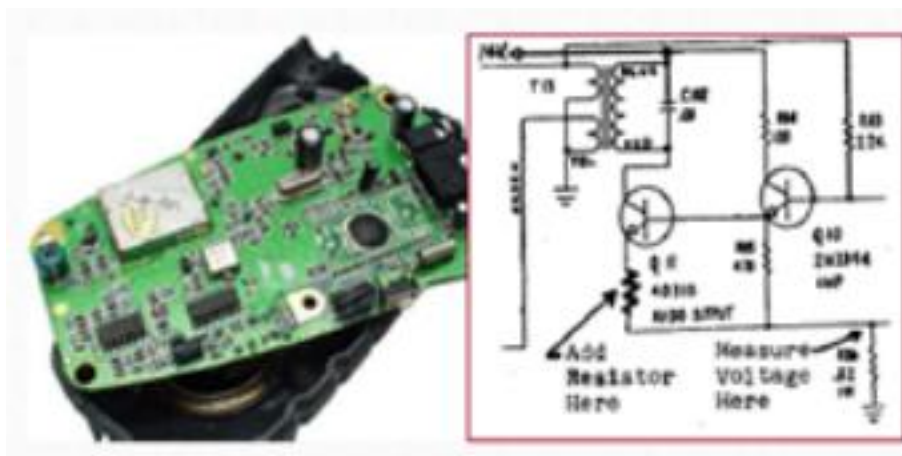
- ❖ Encapsulation means putting together all the **variables** (instance variables) and the **methods** into a single unit called **Class**.
- ❖ It also means hiding data and methods within an Object.
- ❖ A programmer can access and use the **methods** and **data** contained in the **black box but cannot change them**.
- ❖ Use access modifier: **private**, **protected**, **default**.

Section 2

Encapsulation

❖ **Encapsulation:** Hiding implementation details from clients

- ✓ Is the technique of **making the fields in a class private**
- ✓ **Providing access to the fields via public methods.**
 - Prevents the *code* and *data* being randomly accessed by other code defined outside the class.
 - The ability to *modify* our implemented code *without breaking* the code of others who use our code.



❖ Two steps to implement encapsulation feature:

- ✓ Make the **instance variables private** so that they cannot be accessed directly from outside the class. You can only set and get values of these variables through the methods of the class.
- ✓ Have **getter** and **setter methods** in the class to set and get the values of the fields.

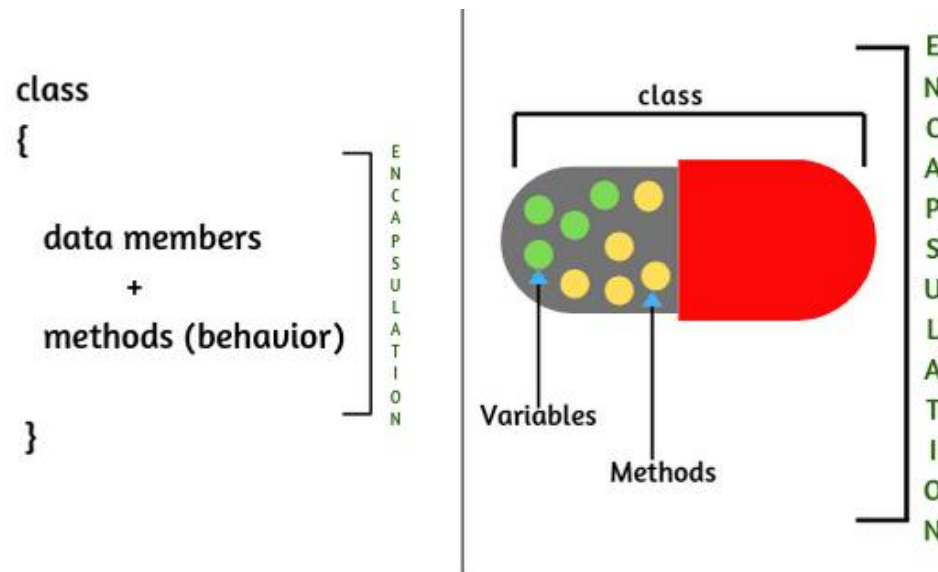
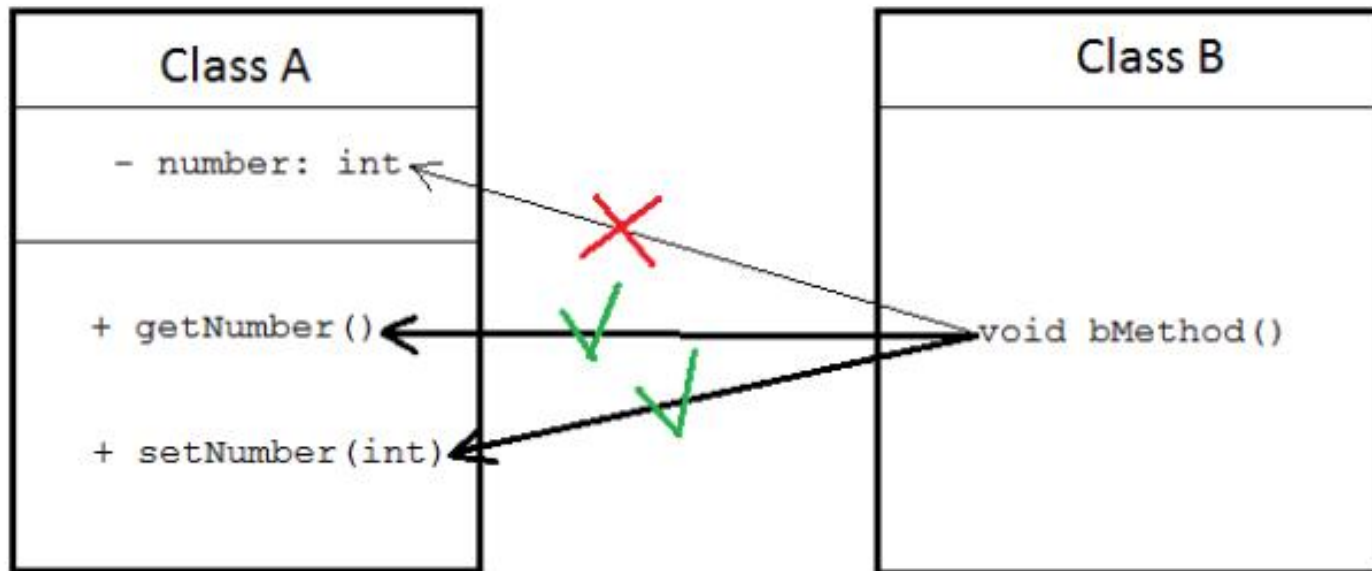


Fig: Encapsulation

Getter and setter method

- ❖ **Getter** and **setter** are two conventional methods that are used for **retrieving** and **updating** value of a variable.



- ❖ The following code is an example of simple class with a private variable and a couple of getter/setter methods:

```
1 public class SimpleGetterAndSetter {  
2     private int number;  
3  
4     public int getNumber() {  
5         return this.number;  
6     }  
7  
8     public void setNumber(int num) {  
9         this.number = num;  
10    }  
11 }
```

- ❖ “**number**” is private, code from outside this class cannot access the variable directly:

```
1 SimpleGetterAndSetter obj = new SimpleGetterAndSetter();  
2 obj.number = 10; // compile error, since number is private  
3 int num = obj.number; // same as above
```

- ❖ Instead, the outside code have to invoke the getter, **getNumber()** and the setter, **setNumber()** in order to read or update the variable, for example:

```
1 SimpleGetterAndSetter obj = new SimpleGetterAndSetter();  
2  
3 obj.setNumber(10); // OK  
4 int num = obj.getNumber(); // fine
```

Why getter and setter?

- ❖ By using **getter** and **setter**, the programmer can control how to variables are accessed and updated in a **correct** manner.
- ❖ Example:

```
1 public void setNumber(int num) {  
2     if (num < 10 || num > 100) {  
3         throw new IllegalArgumentException();  
4     }  
5     this.number = num;  
6 }
```

- ✓ That ensures the value of number is always set between 10 and 100.
- ✓ Suppose the variable number can be updated directly, the caller can set any arbitrary value to it:

```
1 | obj.number = 3;
```

- ❖ The naming scheme of setter and getter should follow *Java bean naming convention* as follows:

getXXX() and **setXXX()**

✓ where XXX is name of the variable.

- ❖ For example with the following variable name:

```
1 | private String name;
```

```
1 | public void setName(String name) { }  
2 |  
3 | public String getName() { }
```

- ❖ If the variable is of type boolean, then the getter's name can be either **isXXX()** or **getXXX()**, but the former naming is preferred.

```
1 | private boolean single;  
2 |  
3 | public String isSingle() { }
```

- ❖ “**this**” keyword in java can be used inside the *method* or *constructor* of Class.
- ❖ It (**this**) works as a reference to the current Object, whose Method or constructor is being invoked.
- ❖ **this** keyword with a **field** and **constructor**:

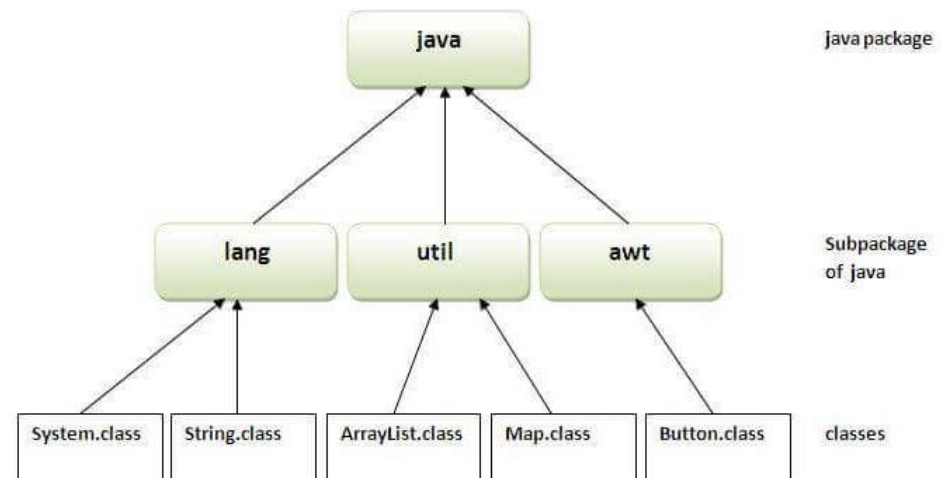
```
3 public class Mobile {
4     private String manufacture;
5     private String operatingSystem;
6     String model;
7     private double cost;
8
9     public Mobile(String manufacture, String operatingSystem) {
10        System.out.println("Constructor with 2 params!");
11        this.manufacture = manufacture;
12        this.operatingSystem = operatingSystem;
13    }
14
15    public Mobile(String manufacture, String operatingSystem,
16        String model, double cost) {
17
18        this(manufacture, operatingSystem);
19
20        this.model = model;
21        this.cost = cost;
22        System.out.println("Constructor with 4 params!");
23    }
24
25    public String getModel() {
26        return this.model;
27    }
28
29    public String toString() {}
30
31
32
33
34
35 }
```

Output:

Constructor with 2 params!
Constructor with 4 params!
Samsung Galaxy S9

```
3 public class MobileTest {
4
5     public static void main(String[] args) {
6         Mobile mobile = new Mobile("Samsung", "Androis", "Samsung Galaxy S9", 2000);
7         System.out.println(mobile.getModel());
8     }
9
10 }
11 }
```

- ❖ A **java package** is a group of **similar types of classes**, **interfaces** and **sub-packages**.
- ❖ Package in java can be categorized in two form: **built-in package** and **user-defined package**.
 - ✓ There are many built-in packages such as *java*, *lang*, *awt*, *javax*, *swing*, *net*, *io*, *util*, *sql* etc.
 - ✓ We will have the detailed learning of creating and using user-defined packages.
- ❖ **Advantage of Java Package:**
 - ✓ Java package is used to categorize the classes and interfaces so that they can be easily maintained.
 - ✓ Java package provides access protection.
 - ✓ Java package removes naming collision.
- ❖ There are three ways to access the package from outside the package:
 - ✓ `import package.*;`
 - ✓ `import package.classname;`
 - ✓ fully qualified name.



- ❖ There are two types of modifiers in Java: **access modifiers** and **non-access modifiers**.
 - ✓ The access modifiers in Java specifies the accessibility or scope of a **field, method, constructor, or class**.
 - ✓ There are **four types** of Java access modifiers:
 - **Private**: The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
 - **Default**: The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
 - **Protected**: The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
 - **Public**: The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.
 - ✓ **Non-access modifiers**: static, abstract, synchronized, native, volatile, transient, etc.

Access Modifiers

Access Modifier	Within class	Within package	Outside package by subclass only	Outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Section 3

Inheritance

- ❖ Inheritance allows you to define a **new class** by specifying only the ways in which it differs from an **existing class**.
- ❖ Inheritance promotes software reusability (tính tái sử dụng)
 - ✓ **Create new class from existing class**
 - Absorb existing class's **data and behaviors**
 - Enhance with **new capabilities**
- ❖ Inheritance represents the **IS-A relationship** which is also known as a *parent-child* relationship.

❖ Syntax:

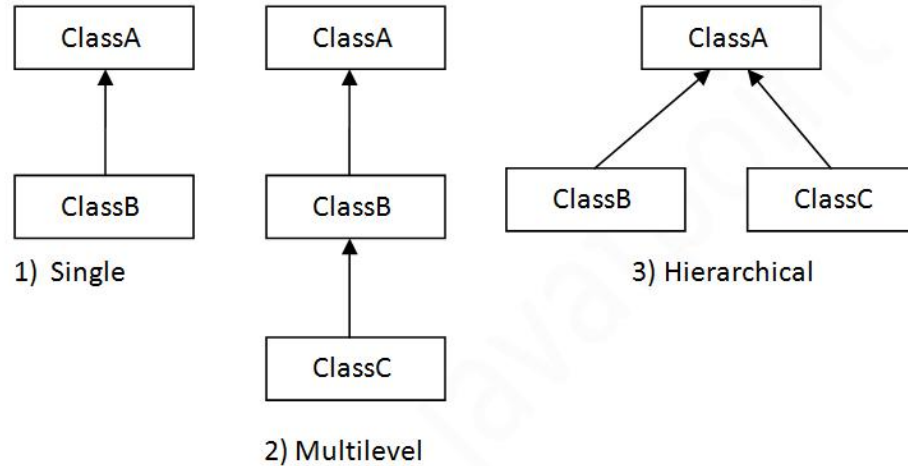
```
class SubclassName extends SuperclassName {  
    //methods and fields  
}
```

❖ Terms:

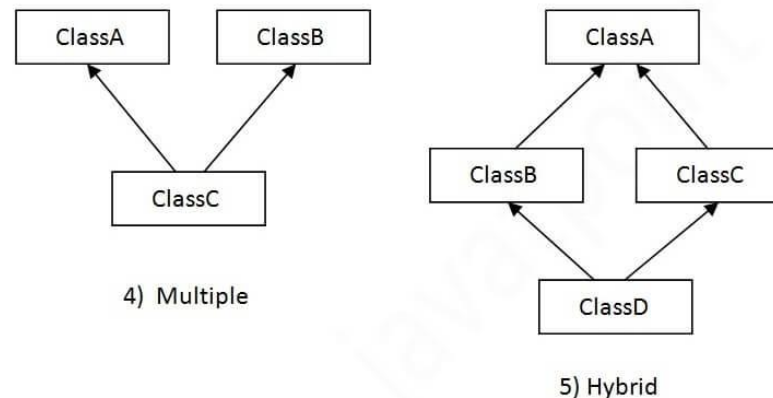
- ✓ **Class:** A class is a group of objects which have common properties.
- ✓ **Sub Class/Child Class:** Subclass is a class which inherits the other class. It is also called a derived class, extended class, or child class.
- ✓ **Super Class/Parent Class:** Superclass is the class from where a subclass inherits the features. It is also called a base class or a parent class.
- ✓ **Reusability:** As the name specifies, reusability is a mechanism which facilitates you to reuse the fields and methods of the existing class when you create a new class. You can use the same fields and methods already defined in the previous class.

Types of inheritance in java

- ❖ Three types of inheritance in java: **single**, **multilevel** and **hierarchical**.



- ❖ *Note: Multiple inheritance is not supported in Java through class.*



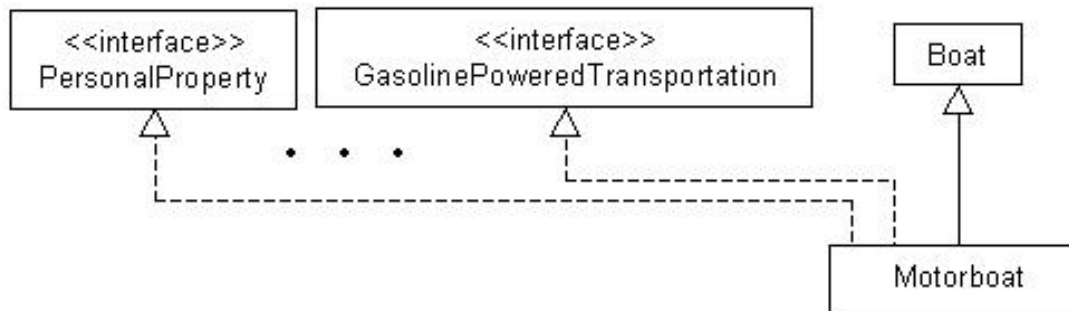
❖ Two kinds:

- ✓ implementation: the code that defines methods.
- ✓ interface: the method prototypes only.

❖ You **can't extend** more than one class!

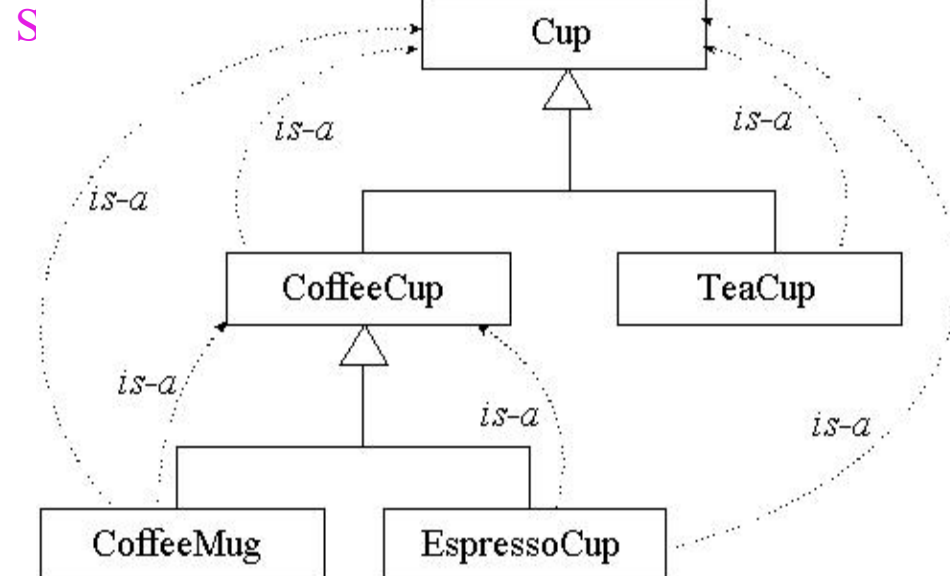
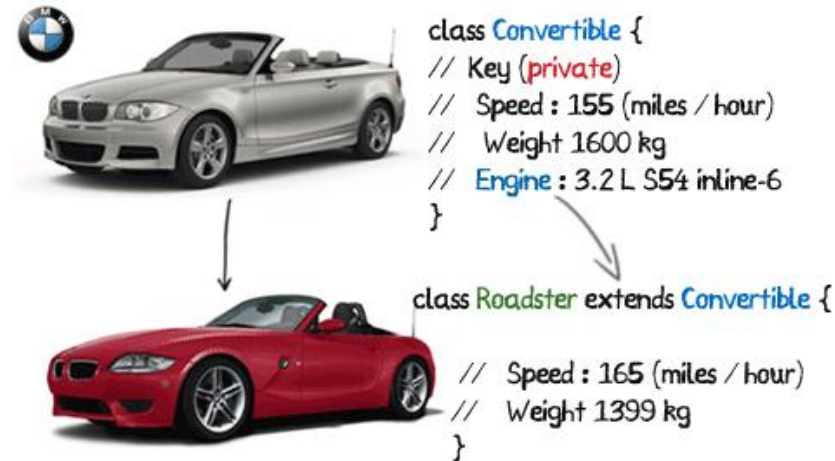
- ✓ the derived class can't have more than one base class.

❖ You can do multiple inheritance with *interface* inheritance.



❖ Inheritance Vocabulary:

- ✓ Superclass/Subclass
- ✓ OOP Hierarchy
- ✓ Overriding
- ✓ "isa" - an instance of a subclass is-a instance of the superclass.



❖ “IS-A”

- ✓ “IS-A” relationship – this thing **is a** type of that thing

- Inheritance
- Subclass object treated as superclass object

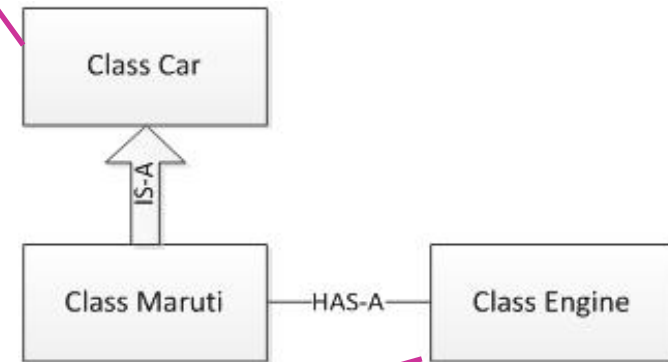
Example: Maruti *is a* Car

- ✓ Car properties/behaviors also Maruti properties/behaviors

❖ “HAS-A”

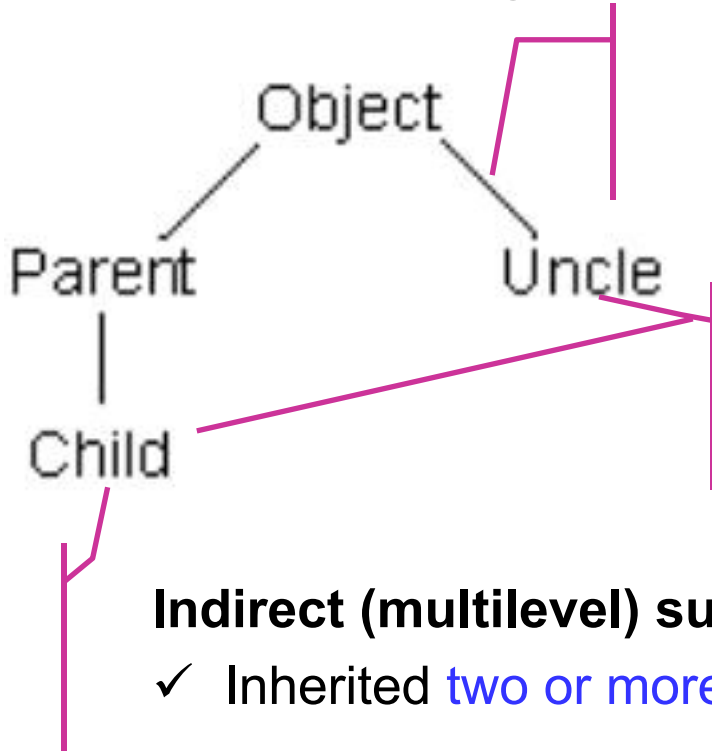
- ✓ “HAS-A” relationship - class A **HAS-A** B if code in class A has a reference to an instance of class B.

- Aggregation
- Object contains one or more objects of other classes as members



Example: Maruti *has a* Engine

■ Class hierarchy



Direct superclass^[kế thừa trực tiếp]:

- ✓ Inherited explicitly (**one level** up hierarchy)

Single inheritance

- ✓ Inherits from **one superclass**

Indirect (multilevel) superclass

- ✓ Inherited **two or more levels** up hierarchy

❖ Multiple inheritance:

- ✓ Inherits from **multiple superclasses**

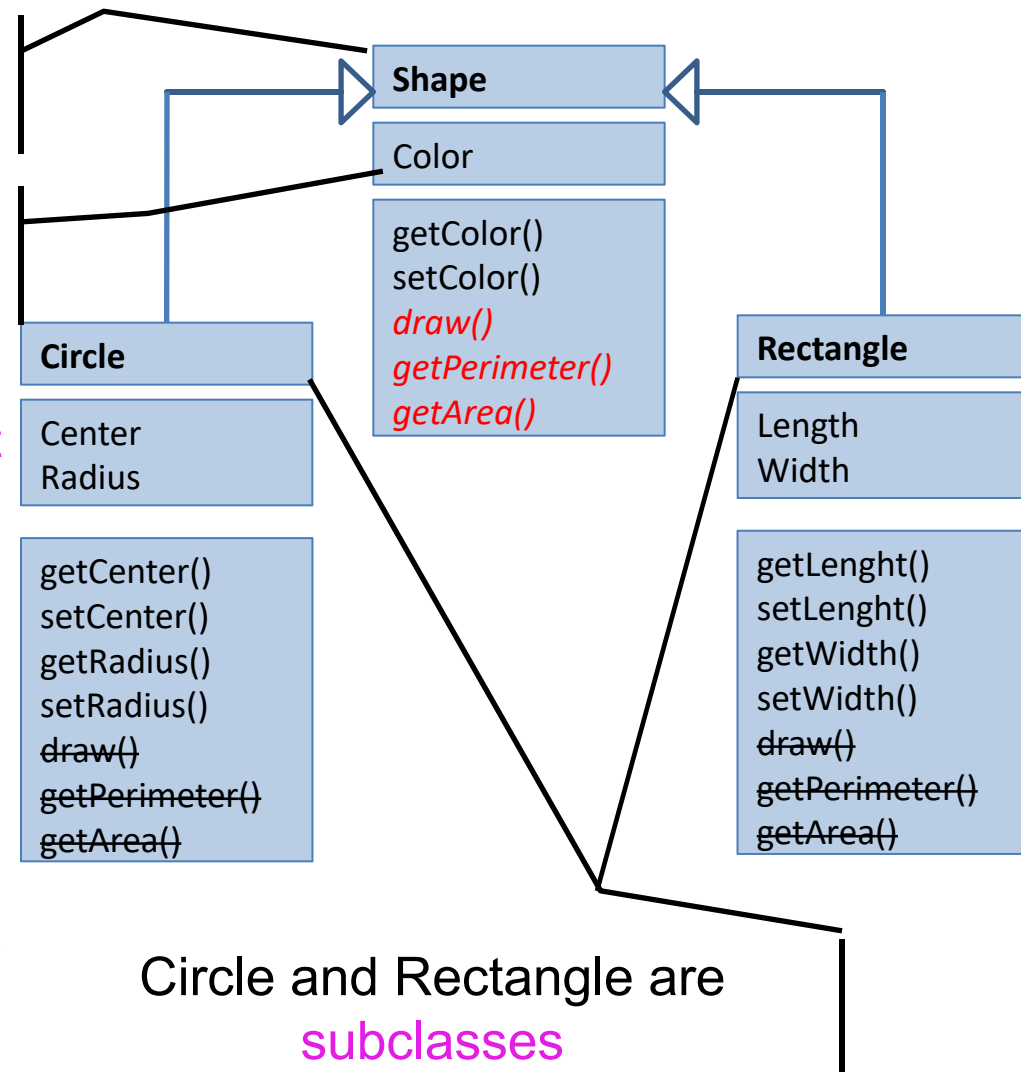
- **Java does not support multiple inheritance in *classes***

Inheritance Example

Shape is **superclass**

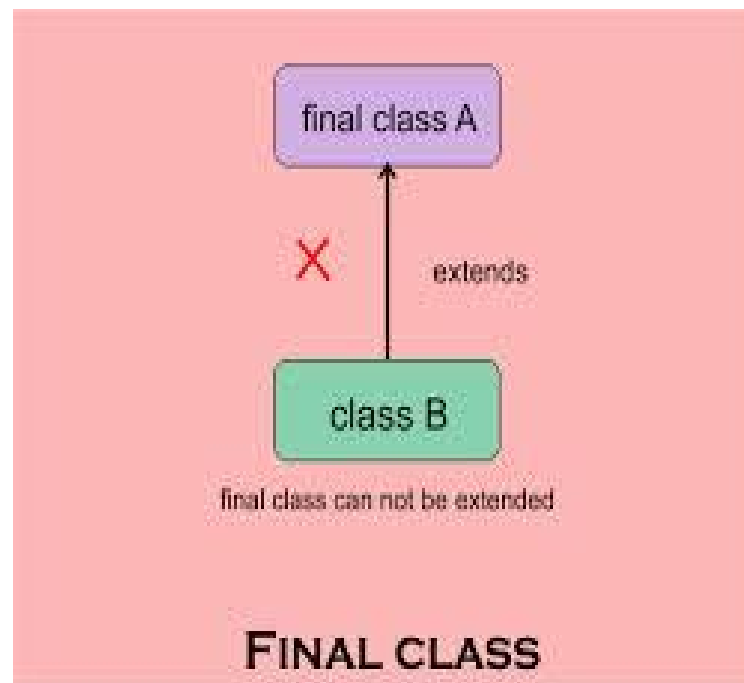
Circle and Rectangle has **Color** property

- Circle **isa** Shape, but Shape **is not** a Circle.
- Method **draw()**, **getPerimeter()**, **getArea()** in Circle overriding method **draw()**, **getPerimeter()**, **getArea()** in Shape.
- If we add/remove property to/from Shape, then it's **affected** to Circle and Rectangle.



❖ Final class:

- ✓ You can declare a class is `final` - this prevents the class from being subclassed.
- ✓ Of course, an `abstract` class cannot be a `final` class.



❖ Instantiating subclass object

✓ *Chain of constructor calls*

- Subclass constructor **invokes** superclass constructor
 - **Implicitly** or **explicitly**
- Base of inheritance hierarchy
 - Last constructor called in chain is Object's constructor
 - Original subclass constructor's body finishes executing last.

❖ Examples:

```
class Building {  
    Building() {  
        System.out.print("b ");  
    }  
  
    Building(String name) {  
        this();  
        System.out.print("bn " + name);  
    }  
}
```

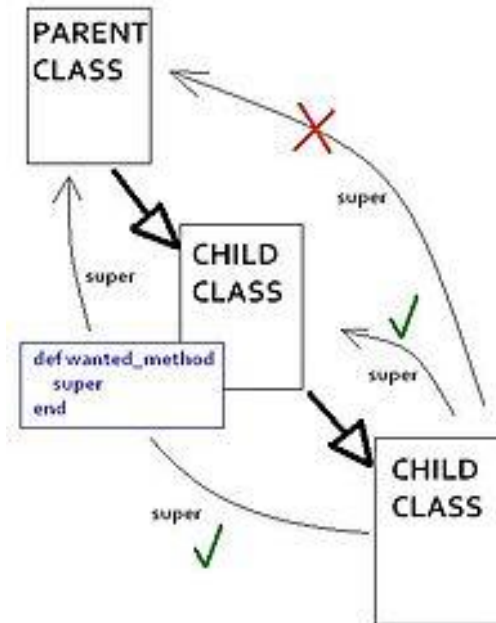
```
public class House extends Building {  
    House() {  
        System.out.print("h ");  
    }  
  
    House(String name) {  
        this();  
        System.out.print("hn " + name);  
    }  
  
    public static void main(String[] args) {  
        new House("x ");  
    }  
}
```

❖ Garbage collecting subclass object

- ✓ Chain of finalize method calls
 - **Reverse** order of constructor chain
 - Finalizer of **subclass called first**
 - Finalizer of **next superclass** up hierarchy next
 - Continue up hierarchy until final superreached
 - » After final superclass (Object) finalizer, object removed from memory

- ❖ Can use **super** keyword to access all (non-private) superclass methods.
 - ✓ even those replaced with new versions in the derived class.
- ❖ Can use **super()** to call base class constructor.

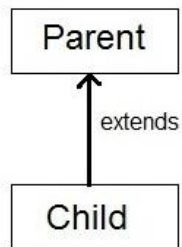
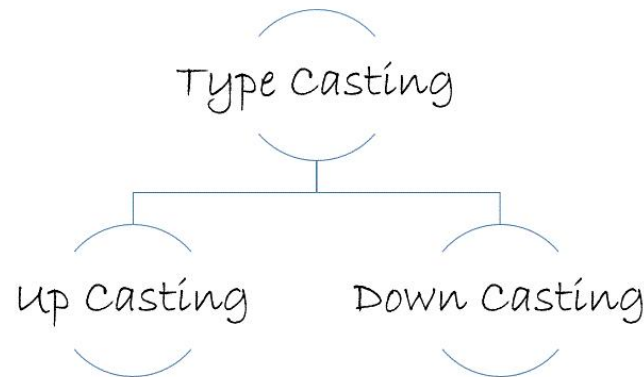
```
class Parent
{
    String name;
}
class Child extends Parent {
    String name;
    void detail()
    {
        super.name = "Parent";
        name = "Child";
    }
}
```



- ❖ Subclass methods are **not** superclass methods

Casting Objects

- Java permits^[cho phép] an object of a **subclass type** to be treated **as an object of any superclass type**. *This is called upcasting.*
- Upcasting and downcasting are **NOT like** casting primitives from one to other.



Parent p = new Child();

Upcasting

~~Child c = new Parent();~~

Compile time error

Child c = (Child) new Parent();

Downcasting but throws

ClassCastException at runtime.

Upcasting is done automatically.

Downcasting must be manually done by the programmer

Casting Objects Examples

```
class Animal {
    public void eat() {
        System.out.println("Generic Animal Eating Generically");
    }
}

class Horse extends Animal {
    public void eat() {
        System.out.println("Horse eating hay, oats, " + "and horse treats");
    }

    public void buck() {
        System.out.println("This is buck");
    }
}
```

```
public class TestAnimals {
    public static void main(String[] args) {
        Animal a = new Animal();
        Animal b = new Horse(); // Animal ref, but a Horse object - upcasting
        Horse h = new Horse();
        a.eat(); // Runs what?
        b.eat(); // Runs what?

        // What is the result?
        Animal c = new Horse();
        c.buck();
    }
}
```

→ Cannot invoke subclass-only (Horse) methods on subclass object through superclass (Animal) reference

❖ **protected** access

- ✓ Intermediate level of protection between **public** and **private**;
- ✓ **protected** members accessible to:
 - superclass members
 - subclass members
 - Class members in the same package
- ✓ Subclass access superclass member
 - Keyword **super** and a dot (.)
 - There is **no super.super....**

❖ Using **PROTECTED** instance variables

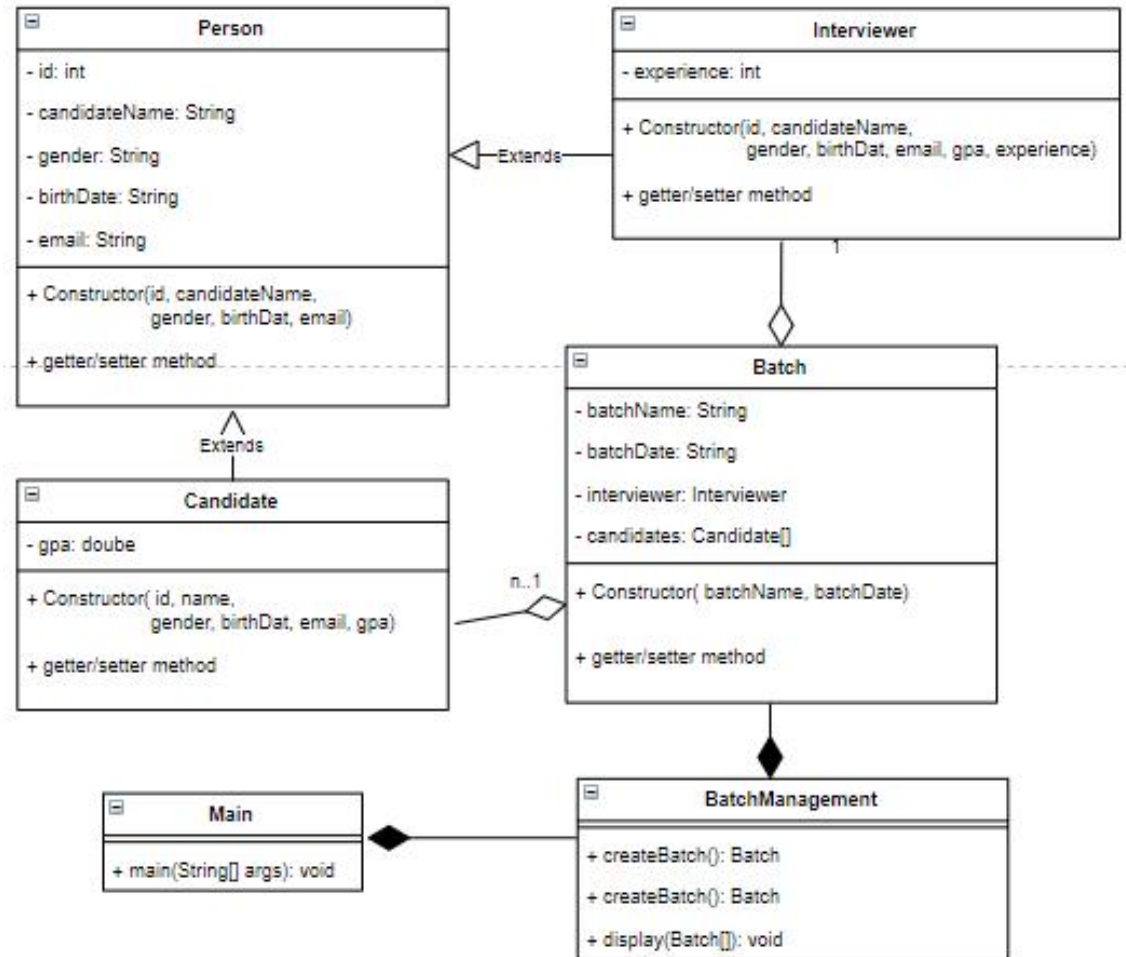
✓ **Advantages**

- subclasses can **modify** values directly
- Slight increase in **performance**
 - Avoid set/get function call overhead

✓ **Disadvantages**

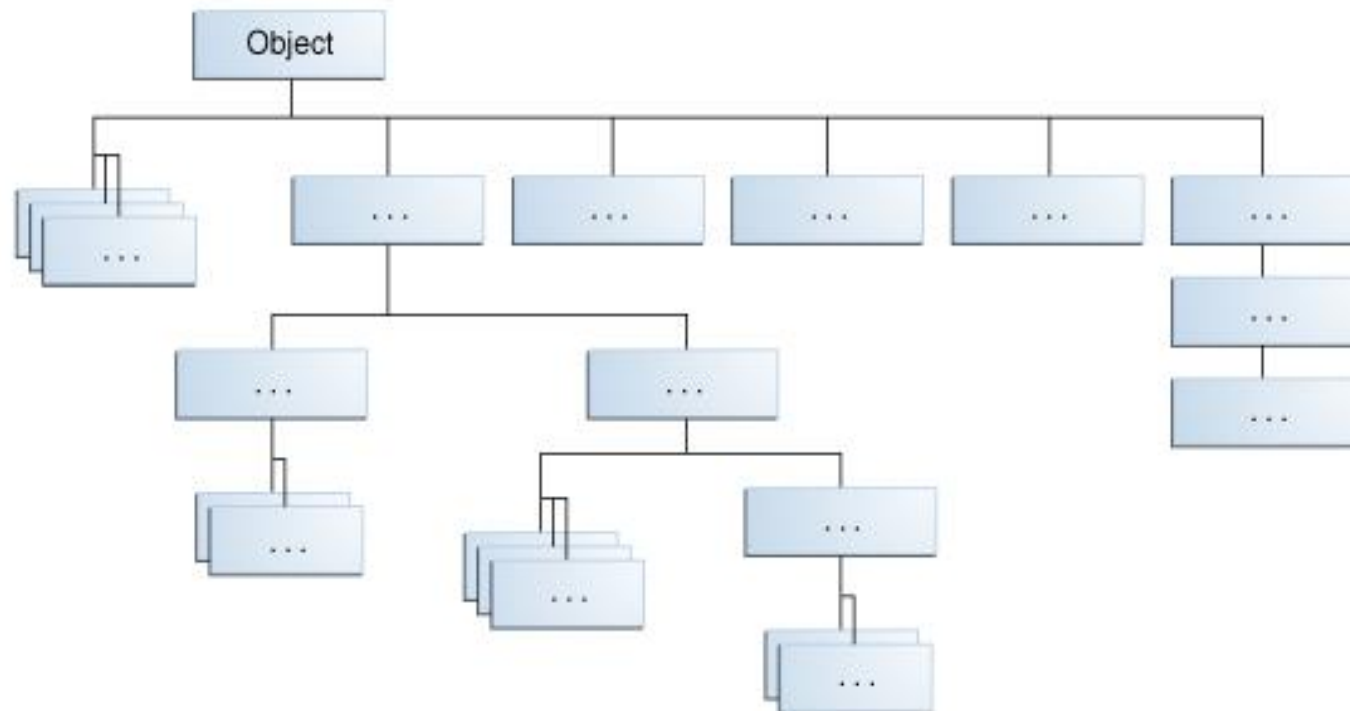
- No validity checking
 - subclass can **assign illegal value**
- Implementation **dependent**
 - subclass methods more likely dependent on superclass implementation
 - superclass implementation changes may result in subclass modifications

- ❖ In class diagrams, as shown in following Figure. Let's implement it using Java:



The class Object

- ❖ Granddaddy of all Java classes.
- ❖ All methods defined in the class Object are available in every class.
- ❖ Any object can be cast as an **Object**.



- ❖ Inheritance is a mechanism that allows one class to **reuse** the implementation provided by another.
- ❖ A class always **extends** exactly one superclass.
 - ✓ If a **class does not explicitly extend another**, it implicitly extends the class **Object**.
- ❖ A superclass method or field can be accessed using a **super.** keyword.
- ❖ Subclass objects **can not access** superclass's **private** data **unless** they change into **protected** access level.
- ❖ If a constructor does not **explicitly** invoke another (this() or super()) constructor, it **implicitly** invokes the superclass's no-args constructor.
- ❖ **Encapsulation:**
 - ✓ Hiding implementation details from clients.

Thank you

